

Prioridades post-separación Backend/Frontend

Estado: separación completada y subida a `feature/motor-reposicion-inteligente` (commit `31a2205`). Ambos proyectos compilan, pero hay 7 ítems críticos pendientes antes de considerar el split listo para producción.

Ordenados por **impacto x esfuerzo**. Los primeros 3 son **bloqueantes** para que la app realmente funcione en runtime; los siguientes son correcciones visuales y de robustez.

P0 — BUGS ESTRUCTURALES BLOQUEANTES (3)

1. Páginas con `redirect()` sin marcar `"use client"` rompen el build CSR

Archivos afectados (5):

- `hammer-frontend/src/app/page.tsx`
- `hammer-frontend/src/app/app/master/timber/trips/page.tsx`
- `hammer-frontend/src/app/app/master/employees/page.tsx`
- `hammer-frontend/src/app/forbidden/page.tsx`
- `hammer-frontend/src/app/unauthorized/page.tsx`

Problema: son los únicos archivos en `src/app/` sin `"use client"` en la cabecera. `redirect()` de `next/navigation` solo es Server-Component-safe; en CSR puro genera comportamientos inconsistentes (el redirect no se ejecuta en hidratación). Las 3 páginas que sólo redirigen quedan **rotas en runtime**.

Fix sugerido (≤30 min):

- `page.tsx`, `timber/trips/page.tsx`, `employees/page.tsx` → marcar `"use client"` y usar `useEffect(() => router.replace(...), [])` con `useRouter` de `next/navigation`.
- `forbidden/page.tsx`, `unauthorized/page.tsx` → añadir `"use client"` (solo renderizan, no usan APIs server-only, pero importan `<Button>` que ya es client).

2. 64 llamadas `fetch("/api/...")` directas, fuera de `apiFetch`

Detección:

```
grep -rn --include="*.tsx" "fetch(" src/components src/app | \
  grep -E '"/api/|`/api/' | grep -v apiFetch | wc -l    # → 64
```

Componentes con mayor concentración:

- `components/users/users-admin.tsx`
- `components/cash-session/cash-session-panel.tsx` (×2)

- components/payroll/employee-manager.tsx (×3)
- components/sales/orders-admin.tsx (×2)
- components/audit/audit-log-viewer.tsx
- components/timber/* (×4)
- components/pos/branch-pos.tsx
- ... más 50 ocurrencias.

Problema: ninguna de esas llamadas pasa por el helper centralizado

@/lib/client/api.ts → apiFetch, por lo tanto:

- **No inyectan** x-csrf-token → todas las mutaciones (POST/PUT/DELETE) devolverán **403 MISSING_CSRF_TOKEN** desde el middleware del backend.
- **No incluyen** credentials: "include" → en producción, cuando la cookie de sesión cruce dominios (si se desactiva el rewrite), las peticiones serán anónimas → **401 NO_SESSION**.
- No comparten reintento al expirar el token CSRF.

Fix sugerido: reemplazo masivo

```
// Antes
const r = await fetch("/api/payroll/history");
// Después
import { apiFetch } from "@/lib/client/api";
const r = await apiFetch("/api/payroll/history");
```

Puede automatizarse con un codemod o `sed -i` cuidadoso por archivo.

3. Estrategia CORS/cookies cross-origin no documentada ni implementada

Estado actual: hammer-api/next.config.ts dice explícitamente

“the frontend talks to this backend via Vercel Rewrites (same-origin), so cross-origin CORS is NOT needed.”

Riesgos en producción:

- Si por cualquier motivo (CDN, custom domain, dev local con puertos separados, app móvil futura) el frontend habla **directo** al backend, ninguna petición autenticará. No hay:
- cabeceras Access-Control-Allow-Origin / -Credentials
- validación de Origin en mutaciones (mitigación CSRF complementaria)
- SameSite=None; Secure en la cookie de sesión (requerido para cross-site con credenciales).
- En desarrollo local, hammer-frontend corre típicamente en :3000 y hammer-api en :4000. El rewrite del next.config.ts cubre este caso **sólo si el frontend se accede vía :3000** — si alguien apunta al backend directo desde el navegador, falla.

Fix sugerido (P0 medio-plazo, P1 inmediato):

- Añadir en hammer-api/middleware.ts un bloque CORS condicional con whitelist desde process.env.ALLOWED_ORIGINS (CSV).
- Verificar SameSite y Secure en la cookie de sesión cuando

```
NODE_ENV === "production" .
```

- Documentar en `DEPLOYMENT_SEPARATION.md` qué variable de entorno controla la lista de orígenes.

P1 — MEJORAS VISUALES PRIORITARIAS (4)

4. No existe `<Skeleton>` ni `loading.tsx` en ninguna ruta

Detección:

- `find src/app -name "loading.tsx" → 0 resultados`
- `find src/app -name "error.tsx" → 0 resultados`
- `find src/app -name "not-found.tsx" → 0 resultados`
- `grep -rln "Skeleton\|Spinner" src/ → 0 resultados`

Impacto UX: después de la conversión a CSR todos los datos llegan **asincrónicamente**. Sin skeletons ni boundaries, el usuario ve pantallas en blanco o “flashes” mientras los `useEffect` cargan. 16/43 páginas manejan `isLoading` localmente — el resto no muestra estado alguno.

Fix sugerido:

1. Crear `components/ui/skeleton.tsx` (≈10 LOC con `animate-pulse`).
2. Crear `app/error.tsx`, `app/loading.tsx`, `app/not-found.tsx` globales con el design system existente (las páginas `forbidden` y `unauthorized` ya muestran el estilo a seguir).
3. Reemplazar los `return null` durante `isLoading` por skeletons.

5. Accesibilidad: sólo 3/88 archivos `.tsx` usan `aria-*`

Detección:

```
Total .tsx: 88
Archivos con aria-*: 3
Archivos con role=: 1
```

Problema: la app es un POS — se usa con teclado, lectores en cajas con monitores pequeños, modo táctil. La densidad de atributos ARIA es extremadamente baja. Botones-ícono (`lucide-react`) sin `aria-label` quedan inaccesibles a screen readers.

Fix sugerido (incremental, no bloqueante):

- Añadir `aria-label` a todos los `<Button>` que solo contienen un ícono (`<Edit/>`, `<Trash/>`, `<X/>`).
- Marcar diálogos y modales con `role="dialog" + aria-modal="true"`.
- Toasts con `role="status"` o `role="alert"`.

6. Backend cross-origin: cookie SameSite y CSRF cookie/header

Relacionado con el ítem 3, pero específicamente visual/UX: si la cookie no se envía, **el usuario verá la pantalla de login en bucle** sin un mensaje claro de “tu sesión expiró”. Hoy ese flujo no existe.

Fix sugerido:

- En `apiFetch`, al recibir un 401 con `reason === "NO_SESSION"`, hacer `router.replace("/login?expired=1")` automáticamente.
 - En `login/page.tsx`, leer `?expired=1` y mostrar toast “Tu sesión expiró, por favor inicia sesión de nuevo”.
-

7. responsive.css aplica min-height: 44px a todos los <a> (rompe layouts inline)

Archivo: `hammer-frontend/src/styles/responsive.css` líneas 5-9.

Problema: `button, a, select, [role="button"] { min-height: 44px; }` fuerza 44px en cada `<a>` — incluyendo enlaces inline dentro de párrafos, badges, breadcrumbs. La excepción `a.inline-link`, `.badge` solo funciona si quien escribe el JSX recuerda añadir esa clase, lo cual **no se está haciendo de forma consistente** en los 88 `.tsx`.

Fix sugerido:

- Invertir la regla: aplicar `min-height: 44px` **sólo** a `<a>` con clase `.touch-target` o dentro de `nav, [role="navigation"]`, no globalmente.
 - O usar selectores más específicos (`a.button`, `a[role="button"]`).
-

Resumen ejecutivo

#	Categoría	Esfuerzo	Bloqueante
1	<code>redirect()</code> en archivos sin <code>"use client"</code>	30 min	✓ Sí
2	64 <code>fetch()</code> directos → migrar a <code>apiFetch</code>	2-3 h	✓ Sí (CSRF)
3	CORS + cookies cross-origin documentadas e implementadas	1-2 h	✓ Sí
4	Skeletons + <code>loading.tsx</code> / <code>error.tsx</code> globales	2 h	⚠ UX
5	Atributos ARIA en botones-ícono y modales	2-3 h	⚠ A11y
6	Auto-redirect a <code>/login?expired=1</code> en 401	30 min	⚠ UX
7	Regla <code>min-height: 44px</code> demasiado agresiva en <code>responsive.css</code>	15 min	⚠ Visual

Total estimado: 8-11 horas de trabajo para dejar el split en estado production-ready y con UX consistente.

Orden recomendado de ejecución:

1. (P0) Ítem 1 — fix rápido, desbloquea redirecciones.
2. (P0) Ítem 2 — codemod masivo, desbloquea TODAS las mutaciones.
3. (P0) Ítem 3 — habilita despliegue real cross-origin.
4. (P1) Ítem 4, 6, 7 — quick wins visuales el mismo día.
5. (P1) Ítem 5 — barrida de accesibilidad incremental.